GroundTruth

The shopping assistant that doesn't lie

Team Crew (TEAM-000802) | Problem R3, Retail track | TalentAP Mega Hackathon 2026 | Status on 30 October 2026

1. The problem

Retailers are rushing to put chatbots on their product catalogues. Most of them **confidently invent products, prices and return policies**. A hallucinated price is a legal and trust problem, not a cosmetic one: the shop has effectively quoted a price it never set.

The task (R3): build a grounded product-discovery assistant over a fixed catalogue that answers natural shopper questions, and **refuses to answer when the catalogue does not support it**.

What the organisers provide	What the solution must deliver
A catalogue of 5,000 products (title, description, attributes, price, stock status, category, ratings)	A working chat interface (CLI, notebook or web page all score equally)
A policy document (returns, shipping, warranty) as plain text	A citation for every factual claim: a product ID or a policy section
150 hidden-answer test questions: answerable, ambiguous, out-of-catalogue ("do you sell iPhones?") and trap questions ("the blue running shoe" when three exist)	An explicit refusal path ("I don't have that") and a clarification path (ask which one, never guess)
A small embedding model and a small instruct model, cached for offline use	An evaluate.py score: groundedness, helpfulness and refusal accuracy, with hallucinated prices penalised heavily
Assumptions: the catalogue is the single source of truth; prices and stock are as of the snapshot; no accounts, orders or payments; shoppers write short, ungrammatical, code-mixed queries	A latency and token-cost report per query. Stretch goal: a multi-turn budget ("under Rs. 2,000, and I already said cotton")

2. How we approached it

The core idea: the assistant never writes free text. Every sentence is assembled in code from catalogue fields, or quoted word for word from the policy, and carries a citation such as [SKU-1234] or [policy:returns]. A price that is not in the source data therefore cannot appear in a reply. This is the opposite of asking a language model to answer and then trying to catch its mistakes.

Each message goes through four steps:

- **Understand** the message with rules: budgets ("under 2k", "1500 ke andar"), sizes and measurements ("65W"), colour, material and brand filters, negation ("not cotton"), superlatives ("sabse sasti"), typos and split words ("smart watch"), Hinglish, and what is being asked (price, stock, rating, a detail, a policy).
- **Find** products with fast inverted indexes and hard constraints: a product must satisfy every filter, so a result can never quietly ignore the shopper's colour or budget.
- **Decide** what kind of reply is honest: one clear match is an answer, several matches for "the" product is a clarifying question, and nothing that fits is a refusal that names the closest cited alternative.
- **Check** the reply with an independent grounding checker that re-derives, from the raw catalogue and policy text, whether every price, number, attribute word and policy sentence is supported. It is shown next to every reply.

Situation	What the assistant does
The catalogue supports one answer	**Answers** from catalogue fields, each sentence cited
Several products fit "the" product (the trap)	**Clarifies**: lists the candidates and what tells them apart, with no price guessed
The catalogue cannot support the request	**Refuses** ("I don't have that"), naming the closest cited alternative when there is one
A policy question	Quotes the matching policy sentences, cited by section; says so when the policy is silent (e.g. Sunday delivery)
A follow-up ("under 2,000, and I said cotton")	Keeps the product type, filters and budget across turns

Why rules, not an LLM. We measured the provided models instead of assuming. The embedding fallback lowered the development score (0.920 against 1.000 on the half we tuned on) because it mapped words to wrong policy sentences. The 0.5B instruct model needed about 3,800 tokens of context and took 13.4 s per call on a laptop CPU, and its JSON was cut off. Neither helped, so both are switched off; the code stays, with timeouts and a fall-back to the rules, for a larger model. The shipped path makes zero model calls.

3. What we built, and the results so far

Brief requirement	Our delivery
Chat interface	A command-line chat and a web page (product cards, citation chips, compare, retailer view)
Citation on every claim	Every claim sentence is cited; all 72 answered test questions carry citations and the structured fields match the text (refusals make no claims)
Refusal and clarification	Three reply types; on the 150 test questions: 72 answers, 32 clarifications, 46 refusals
evaluate.py score	Our own scorer (groundedness, helpfulness, refusal accuracy, a heavy penalty per hallucinated price) and the organisers' own scorer, run unchanged
Latency and token cost	Per-query latency and tokens in every prediction line and in a report; the shipped path uses 0 model tokens
Stretch: multi-turn budget	Works: material, gender and budget carry across turns

Measure	Result	Note
Official scorer, 20 dev questions never tuned on	**0.900**	shipped baseline 0.8275
Official scorer, all 40 dev questions	**0.950**	baseline 0.7315; includes the half we tuned on
Stress set built from the catalogue, held-back half	**1.000**	81 questions written by us
Shopper-wording set, second holdout (310 questions)	**0.733**	wording outside our word lists; 0.509 before the latest fixes
AI-written shopper-style questions (64)	**0.556** first contact	an AI wrote them, so this is not a real-shopper measure; 0.953 after we fixed what they showed (used, not evidence)
Need and occasion requests (50, AI-written), half held back	**0.68**	0.06 before the needs map
Real shoppers' questions	**not measured**	the brief's Task 4: still to do, with real people
Hallucinated prices	**0**	in every run we made
Speed and model use	**about 0.6 ms** per query, **0** model tokens	an LLM writing every reply: about 13 s, 3,950 tokens per query (an estimate from one measured call)
Automated tests	**251** pass	Python 3.9 and 3.12; lint and formatter clean

These scores are on small samples and on sets we wrote ourselves, measured on 29 and 30 October with the real data pack. The zip must be rebuilt from the final commit before upload, and the hidden test score may be lower.

4. Features beyond what the brief asks for

The brief needs a working chat and an honest scorer. We added the following so the assistant is also something a retailer would trust and a judge can inspect in 90 seconds.

For shoppers

- **Citation chips that open the source.** Tapping a citation flips the card to the exact catalogue record, or opens the policy section with the quoted sentence highlighted.
- **A live "sources checked" badge** on every reply, from the same independent checker the evaluation uses.
- **Product cards built only from cited records**, a side-by-side **Compare**, and a panel showing the filters and budget it remembers and how it read the last message.
- **Needs and occasions:** 'something for my morning workout' or 'mummy ka birthday' is answered from a hand-written map of needs to kinds of product, and a gift with nothing else to go on asks who it is for.

- **Offline voice input (optional):** a Speak button turns speech into text on the computer with the Whisper base model, then uses the same grounded path as typed text. 17 of 24 spoken test questions matched the typed answer.
- **Honest about silence:** it says when the policy does not cover something (Sunday delivery) or the catalogue does not list a fact (water resistance), instead of guessing yes or no.
- **Real shopper language:** Hinglish and Telugu-English, misspellings, split product names, measurements ("65W charger"), negation and "cheapest / best rated" phrasing, and brands the shop does not carry are refused with cited alternatives.

For the retailer

- **A retailer view (/insights):** how questions were answered, speed, replies that failed the check (target 0), what was asked for that nothing in the catalogue fits (kept apart from words the assistant could not place, so a typo is not mistaken for a missing product), which questions needed clarifying, and a CSV download.

For trust and operations

- **The grounding checker is stricter than the brief needs:** each price must belong to the product beside it, the shopper's own number is never accepted as a price, attribute words must come from the cited record, and policy sentences must be verbatim.
- **Strict catalogue loading:** a malformed row, a duplicate ID or a wrong product count stops the run instead of silently dropping products.
- **Shipping time is never read as stock:** "ships in 3 days" leaves the stock status unknown.
- **Built for many users:** read-only shared index, a bounded per-user session store, request validation, a same-site check, rate limits and security headers; about a thousand queries per second per process on one laptop (more users means more worker processes, as Python threads share one interpreter lock).
- **Tooling:** a stress set, a shopper-wording set with honest holdouts, a code tour for explaining the code, a per-query cost report with a clearly labelled LLM baseline estimate, a submission builder that checks the organisers' checklist, and a 90-second demo script.

Planned next (shown as the roadmap in the web page, not built)

- Voice questions in Hindi and Telugu, answered with the same citations (a first, English-leaning voice input exists)
- Search by photo, matched only to catalogue items
- Live price and stock sync, replacing the fixed snapshot date
- Price-drop alerts, and size help from the catalogue's own size charts

5. Limitations and problems we met

What we found wrong along the way, and fixed:

- **Our own tests flattered us.** The assistant scored 1.0 on sets written alongside it, but 0.827 on 60 independently written shopper questions. The fixes took a held-out set from 0.769 to 0.942, and the synthetic sets are now treated as regression checks only.
- **The price checker could be fooled** by a price that was right for another cited product, or one only the shopper had typed. Now fixed; the assistant's own replies never made the mistake, so this strengthened the safety net.
- **"Ships in 3 days" was read as "in stock".** Fixed: shipping times now leave stock unknown.
- **The retailer view overclaimed:** a typo was counted as a product the shop does not stock. Fixed by separating the two.
- **Catalogue rows could be dropped silently,** and a Windows connection reset made one test flaky. Both fixed.
- **Answers showed raw field names** ("The water_resistant of ... is False") in 43 of 150 test answers. Now readable ("water resistant: No"); 3 remain, all word-for-word policy quotes.
- **Our new attribute-word check flagged 36 of 190 real replies** when first run on the real pack (attribute names matched attribute values such as "color pencil"). Fixed; it now flags 0 of 190.
- **We mislabelled AI-written questions as real shoppers.** Caught on 30 October and corrected everywhere: the scores stay as measurements of AI-written questions, and Task 4 is marked not done.
- **The provided models did not help** (section 2), so they are off.

What is still limited:

- **Small samples.** Official scores come from 20 and 40 development questions; the hidden 150 questions may score lower.
- **Unusual words are refused, not guessed.** With no model in the shipped path, a synonym the rules have never seen ("jogging") may be refused. That is safe, but sometimes unhelpful.

- **Unseen shopper wording is our biggest weakness.** On 310 shopper phrasings whose wording appears in none of our word lists, the score is **0.733** (it was 0.509 before the latest fixes). Word lists only help the words they contain, and the needs map is hand-written, so a need outside it (such as 'make my daily routine easier') is refused.
- **We have no measurement from real shoppers yet.** The 64 questions first called Task 4 were written by an AI; we found this on 30 October and corrected our documents. The honest test is questions typed by real people, with a record of who typed each one.
- **Voice input is early:** 17 of 24 on synthetic voices, Hindi words are often misheard, and browsers allow the microphone only on localhost or HTTPS, so it does not work from a phone on plain HTTP.
- **Brands are a hand-written list.** 124 more brands are now refused by name, but an unlisted brand can still be ignored and other brands' products shown, with a small note.
- **"Which is better?"** between two kinds of product gets a clarifying question, because the catalogue has nothing to judge "better" by. The policy-silent detection covers delivery days and times only.
- **Still to do:** collect questions typed by real people (with a record of who typed each) and score them once, look at the demo on a real phone, and rebuild and upload the zip from the final commit.